MASARYK
UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Brno, Spring 2026

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Musikk: A Modern, Social-First Music Streaming Platform

Bachelor's Thesis

KIRILL VOROZHTSOV

Advisor: Mgr. Luděk Bártek, Ph.D

Brno, Spring 2026


FACULTAS ARTIS INFORMATICÆ
UNIVERSITAS MASARYKIANA

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, the following AI tools were used:

- **ChatGPT**: Used for debugging and code improvements, as well as LaTeX formatting.
- **Grammarly**: Used for text correction.

Kirill Vorozhtsov

Advisor: Mgr. Luděk Bártek, Ph.D

Acknowledgements

I would like to thank Mgr. Luděk Bártek, Ph.D., for his time and the valuable advice he provided for this thesis.

Abstract

This bachelor's thesis aims to develop a web-based music streaming platform focused on social interactions between users.

The theoretical part presents a survey that analyzes people's music listening habits, showing that such a platform could attract a solid user base. It then compares existing music-related services with built-in social features and derives functional and non-functional requirements for the proposed system. Then, based on these requirements, suitable technologies and architectural patterns are selected.

The practical part covers the implementation details of the platform, including data model, audio processing, API design, and representation layer.

The result is a fully functional and deployed application.

Keywords

Audio Streaming, Python, JavaScript, MPEG-DASH, HLS, WebSockets

Contents

List of Tables

List of Figures

1 Introduction

Music streaming has been steadily growing in popularity each year for the past decade. [1] And this is to be expected; numerous studies have demonstrated how important music is for people. For instance, Rentfrow [2] suggests that music can influence mood, change self-perception, and help with social bonding. In addition, the rise of the clubbing scene, music festivals, and other social music-related activities clearly indicates the important social role of music. [3]

However, despite this social significance, it is surprising that features which facilitate social interactions are still not widely implemented in the existing music streaming platforms, as will be shown in **Chapter 3**.

The goal of this thesis is to design a music-centric platform that provides capabilities for collaboration and social interaction around music.

This work is divided into the following **six** chapters:

- **Chapter 2** Presents the outcomes of a survey illustrating how people consume music, how prevalent it is in social interactions, and why this type of platform can be relevant.
- **Chapter 3** Explores which social features existing streaming platforms and other music-related services already offer.
- **Chapter 4** Outlines the functional and non-functional criteria for the application.
- **Chapter 5** Describes the choices of technologies that are used by the application.
- **Chapter 6** Explains the development process and implementation details.
- **Chapter ??** Summarizes the results of the work.

2 Music Consumption Survey

In order to better support the choice of topic and demonstrate that a music streaming platform centered on social interactions could be a relevant service, a brief survey was conducted. It examined individual music listening and discovery habits, platform usage patterns, music-related interactions between people, and attitudes towards potential social features on the platform.

2.1 Methods

The service chosen for the questionnaire was Google Forms [4]; it provides a simple interface for survey creation, allows for easy sharing of the form, and supports extracting results as CSV files.

The questionnaire consists of 15 questions. Most of them are multiple-choice and closed-ended, with some allowing a custom answer. All custom answers are grouped under the "other" category in the provided figures. Answers in their original form can be found in the full survey.

The complete questionnaire is included in the thesis repository under `text/extras/survey-questions.pdf` and online.[1]

2.2 Results

This section presents the survey outcomes and analyzes their significance. Not all questions are discussed; for example, some questions dedicated to music discovery are omitted.

A total of 119 people participated, with the majority being in the 18 to 30 age group, with the exact distribution shown in Figure 2.1.

1. `https://docs.google.com/forms/d/1vhhAu_SfuHV4xy6JoDaRsM5uCElYn_csTmN8u4ZlpHc`

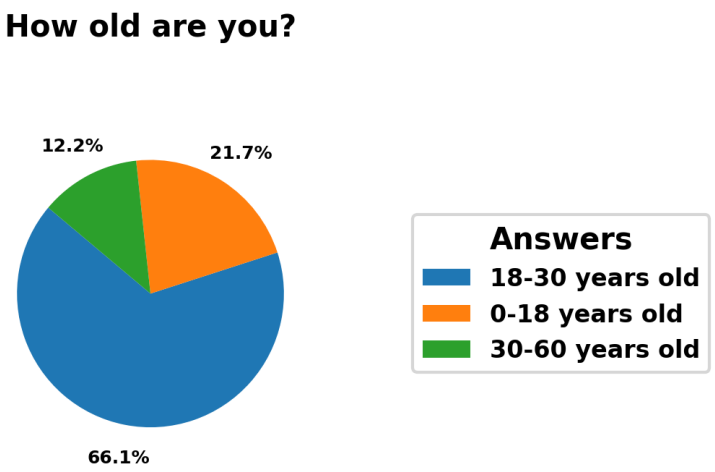

Figure 2.1: Age Distribution of Respondents

2.2.1 Engagement

Firstly, it was necessary to determine whether respondents listen to music frequently and whether they use streaming platforms for this purpose.

Over 50% of respondents reported listening to more than 500 songs per month, i.e., 16 songs a day on average (Figure 2.2), and nearly 80% stated that they listen to music daily (Figure 2.3).

The numbers clearly indicate that people have a significant interest in music, and the next logical step was to determine how the audio content is predominantly accessed.

2.2.2 Listening Methods

As shown in Figure 2.4, the proportion of respondents using streaming platforms accross all age groups is approaching 100%, with slightly higher numbers in the younger age groups. Although downloaded files and physical media are still used, especially among people aged 30-60, they are usually only a secondary option next to streaming solutions.

The question dedicated to the popularity of individual platforms has shown that Spotify is the most used one, which aligns with global

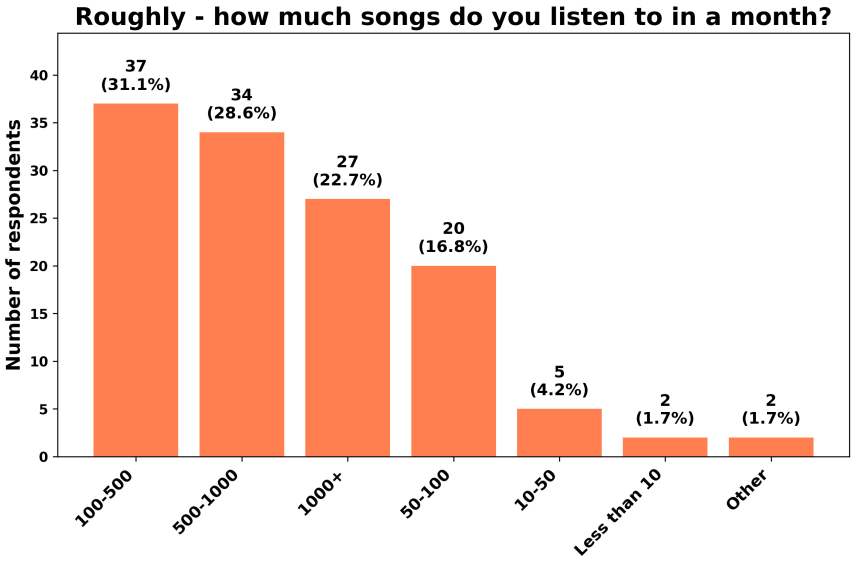

Figure 2.2: Monthly Song Count per Respondent

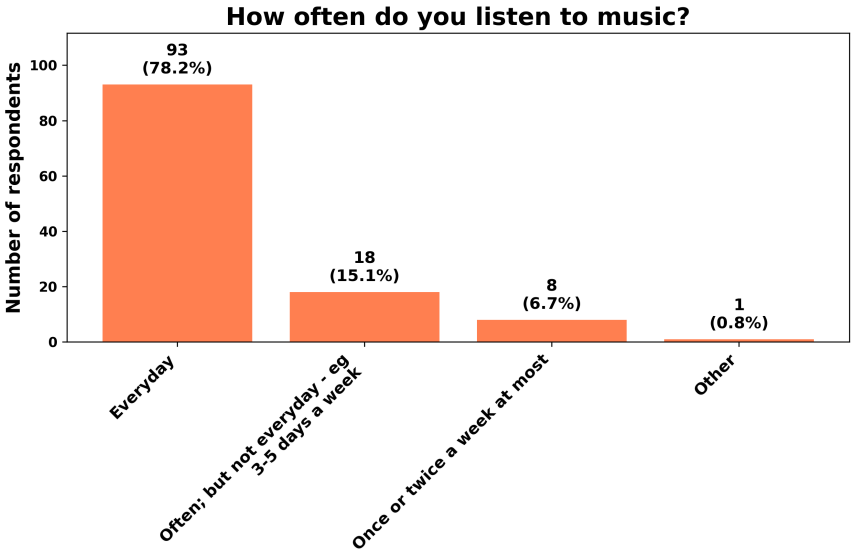

Figure 2.3: Listening Frequency of Respondents

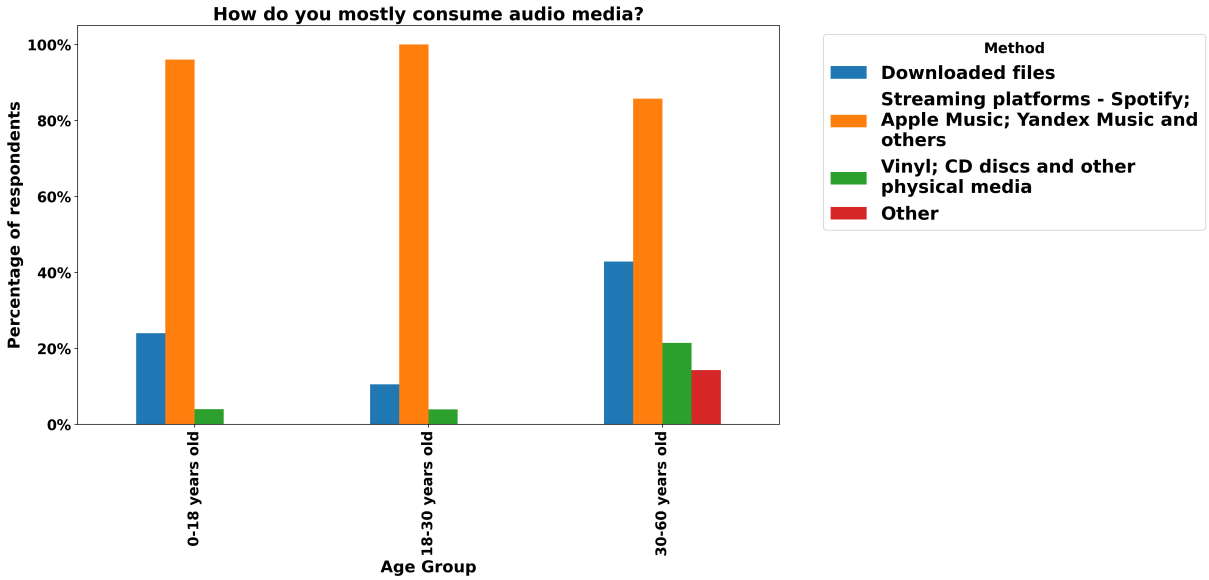

Figure 2.4: Audio Media Consumption by Age Group

statistics [5]. However, Yandex Music, coming in second place, deserves further explanation. Most of the respondents were from Russian-speaking countries, specifically Russia, and, with many Western companies leaving its market in 2022, including music streaming services, most users have moved to locally available products: Yandex Music, VK Music, and Zvuk. This is also the reason, why other popular region-specific platforms, such as Amazon Music (US), QQ Music (China), and JioSaavn (India), are absent from the survey.

The complete statistics are shown in Figure 2.5.

The survey showed that streaming services are indeed widely used. The next important step was to analyze the regularity of music-centered social interactions.

2.2.3 Social Interactions

The results of Question 10 indicate that nearly all respondents (99.2%) reported listening to music with others at least sometimes, primarily in offline settings such as gatherings or car rides (Figure 2.6). Online collaborative listening methods, although less common, were also mentioned by a quarter of respondents.

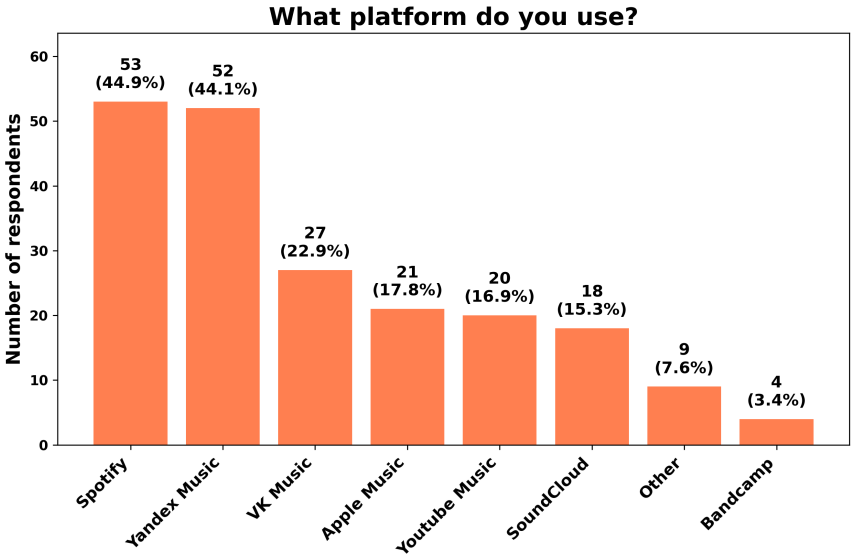

Figure 2.5: Streaming Platforms Popularity

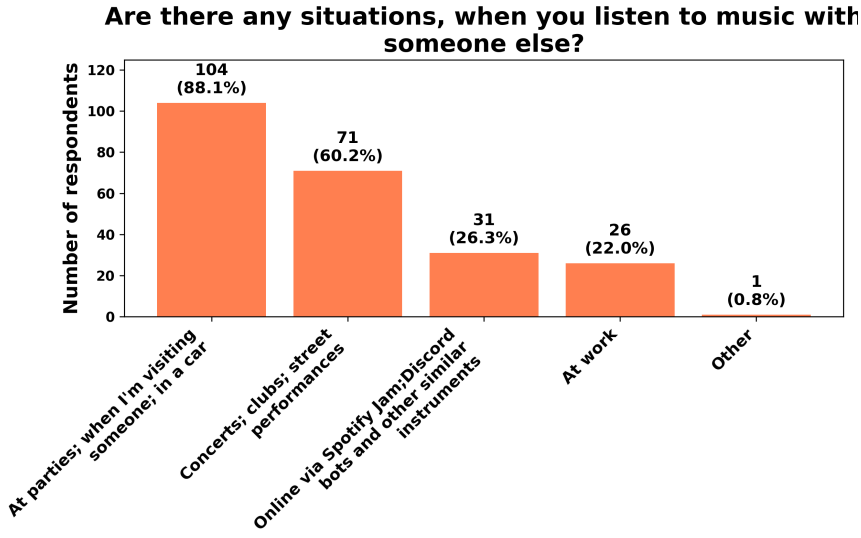

Figure 2.6: Social Interaction Methods

Question 11 (Figure 2.7) examines the frequency of these interactions: the results show that a significant portion of respondents engage in shared listening regularly. Around 60% reported doing so at least a few times per month, with a smaller group indicating almost daily interactions. This supports the previously mentioned idea in **Chapter 1** that music consumption is a common social activity.

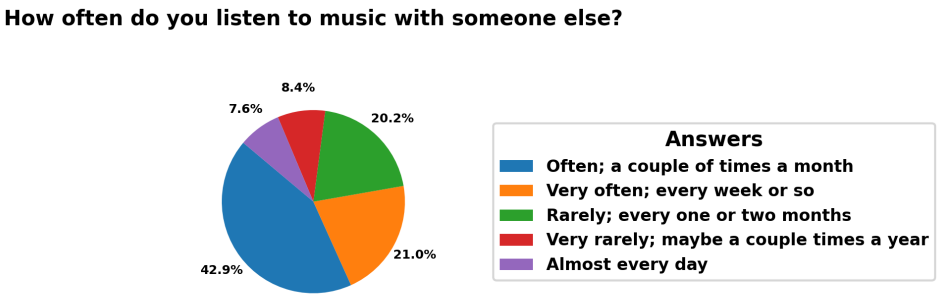

Figure 2.7: Social Interactions Frequency

The next question is phrased as follows: "How often do you/your friends share/discuss music?". While the previous question focused on real-time in-person collective listening, this one highlights more intentional and in-depth musical interactions, such as sharing songs, recommending music, or discussing it. Music, in that case, is not just the background but the primary purpose of engagement. The same trend as before can be observed in the responses: over 70% (Figure 2.8) indicated that they share music at least several times a month, with many doing so weekly or more often. This further emphasizes the active social role of music.

Question 13, "How does that happen?" (in terms of music sharing), asks about the specific ways people share music. Many respondents (Figure 2.9) stated that they typically send links from streaming platforms through external messaging apps. While this shows an apparent demand for sharing music online, it also reveals a gap among platforms, as these interactions remain fragmented. Hence, enabling such exchanges directly within the streaming app could provide a smoother and more uniform experience.

Responses to the question "Would you say that you know a lot of people with similar music taste?" were mixed, with a near-even split.

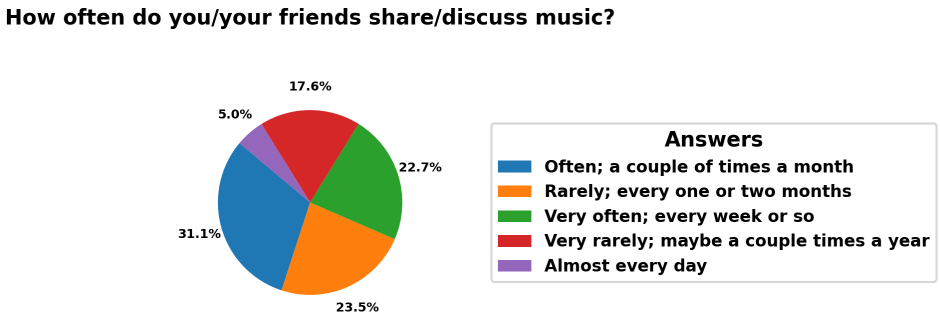

Figure 2.8: Music Sharing Frequency

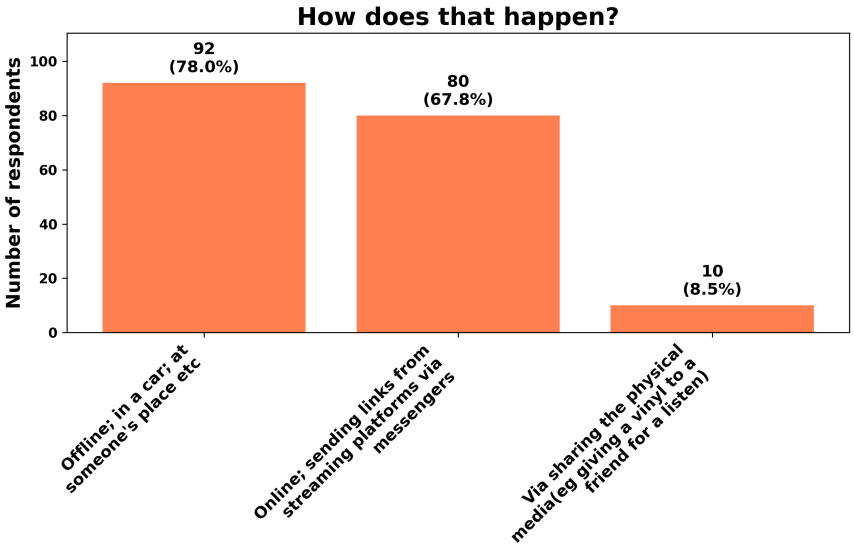

Figure 2.9: Music Sharing Methods

This suggests that while some users already have a social circle with similar music taste, many do not. In addition, when asked "Would you like to connect with others who share your musical taste, or influence those around you to explore and appreciate the music you enjoy?", the majority answered positively.

This data further support the idea that social discovery features could be useful within a streaming platform. Moreover, over 60% of participants stated in Question 15 that they would use additional social features if available on a streaming platform.

2.3 Summary

Overall, the results indicate that people listen to music frequently, with streaming services being their primary medium of accessing it. The data also suggest that music plays an important role in everyday life and often appears in social situations.

Yet, existing platforms only partly support interaction between users. This makes the idea of a music streaming service with built-in social features both relevant and likely to address real user needs.

3 Existing Platforms

In this chapter, existing platforms and services are examined to gain a better understanding of the instruments people use when interacting with music. Additionally, this chapter provides insight into the potential features that could be implemented in the resulting application.

Firstly, streaming services are discussed, as the main aim of the thesis is to create a system for music streaming. Then, other music-related services (e.g., forums) are presented, as they offer alternative possibilities for engaging with audio content that may be beneficial for the future platform. Lastly, platforms such as YouTube and TikTok are examined, because the survey has shown that they are a popular way to interact with music 3.1.

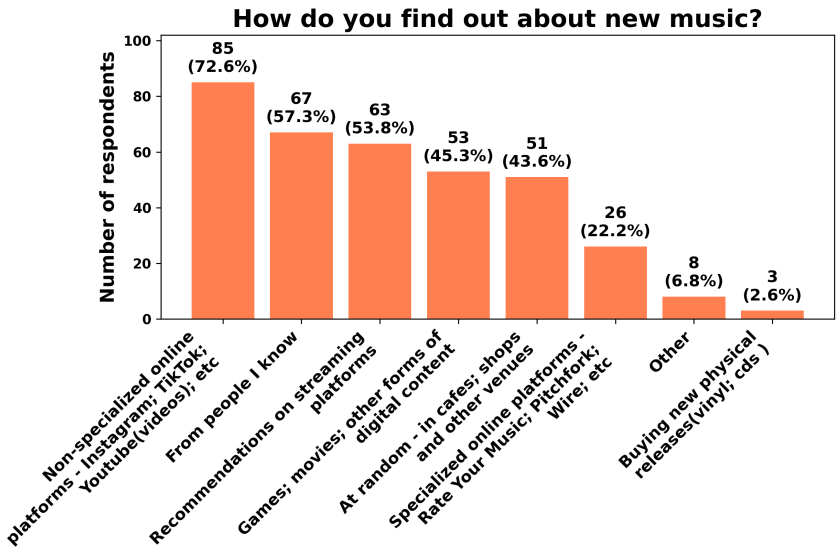

Figure 3.1: Music Discovery Methods Popularity

3.1 Streaming Services

As already illustrated in Figure 2.4, and further confirmed by the study of the International Federation of the Phonographic Industry [6], the

most common way of music consumption and discovery is through streaming services. Although many options are available, this section only considers those that are both popular and have unique features. The descriptions, rather than offering a general portrayal of each platform, will focus on the service-specific social features.

3.1.1 Spotify

One of the most prominent social features on Spotify is the "Spotify Jam" feature [7]. It lets people create a collective song queue that is then synchronized among all connected users. Moreover, volume, the order of songs, and other playback aspects can be controlled individually by each user.

Another notable feature is "Friend Activity" [8], the followers of a given user are currently listening to.

The "Listening Parties" feature also deserves a mention: these are live chats with a limited capacity that can be joined for a short time when new music is being released [9, 10]. This somewhat brings the offline collective music discovery experience to the online world.

During the implementation of this thesis (the text was initially written in spring–summer 2025), Spotify has also added support for user-to-user chats (from September 2025) [11]. This further supports the claim that social features are in demand on music platforms.

3.1.2 VK Music

VK Music is a music service integrated into the VK social network. Consequently, it is possible to send songs and playlists via private messages or add them to posts in groups. VK Music also provides an "Updates" tab, which is populated by the audio materials that any followed user or group has added to their "Liked Feed" [12].

3.1.3 SoundCloud

SoundCloud is one of the few platforms that lets its users leave comments and reactions on songs and playlists [13, 14]. In addition, each user has a feed consisting of personal uploads and reposts of others' content [15], which is visible when visiting their profile.

3.1.4 Bandcamp

Every Bandcamp user has a profile with four tabs: “collection”, “wishlist”, “followers”, and “following”. The most interesting feature is under the “following” tab: users can subscribe to available genres and then specify the ones they want to showcase to others viewing their profile. The “wishlist” tab is also noteworthy, as in this tab the users can showcase what they are looking forward to listening to in the future.

This concludes the streaming services section. Services that are focused on music, but not on the streaming itself, are discussed next.

3.2 Forums, Blogs, and Other Music Media

Different resources are gathered under these umbrella terms, from professional music review websites to amateur and personal pages. Again, only widely used services that offer something noteworthy are listed.

3.2.1 Rate Your Music

> “Rate Your Music is one of the largest music databases online. It is an incredible tool that can help you find and learn about new music to listen to.” [16]

This website is one of the biggest platforms with user-created content. Every member can write reviews and set ratings for music releases, contribute to the “wiki” consisting of genres, thematic lists, and charts, and connect with other members of the forum.

3.2.2 last.fm

Last.fm is primarily a listening statistics and music discovery service. However, it also has a couple of notable social features:

1. A global and personal events pages. In the former, users are able to browse the upcoming events near their location, and in the latter they are able to showcase which events they plan

to attend. Other users can then see, either on the event page or on that user's profile, which events the user is interested in and/or plans to attend.

2. A "Neighbours" tab. The "Neighbours" tab shows other users with similar listening habits, based on shared artists and an overall compatibility score.

3. "Shouts" section. The "Shouts" section is a profile-scoped feed, where the users can leave comments and reply to what others have written.

This section has introduced a couple of possible features not previously seen in the listed streaming services, such as discussion forums and profile feeds.

The following section examines platforms that are not primarily focused on music but still influence how people engage with it.

3.3 General-Purpose Platforms

As mentioned earlier, one of the most interesting results of the survey is the high number of people discovering music on platforms that are not focused on music. Services like Instagram, YouTube, and TikTok, which are primarily used for sharing videos and other user-generated content, have become closely tied to how people find music and listen to it. In recent years, these platforms have started to promote music more directly by allowing users to embed tracks within their platform-native content.

Instagram, for example, added a special audio tool that lets users include music in their short videos ("Reels" [17]), helping songs reach more people through visual content.

YouTube automatically identifies songs used in videos and displays them in the description, making it easier to find and listen to them.

TikTok has had an even bigger impact: many songs have gone viral and later became hits in charts and streaming apps because people use them in popular videos (these are often called "TikTok Hits").

This shows that big platforms are not just reacting to users' interest in music, they actively help promote it by connecting songs with content that people want to watch and share.

An integration with such services can be a big plus for the platform. A possible feature could be a badge stating "This track was in a TikTok user Alice123 sent you today!".

Summary

The table 3.1 summarizes the comparison between the features mentioned in the sections above.
Difficulty suggests the possible complexity of implementation.
Mode assumes the typical context in which the feature operates.
Social Interaction Level indicates user involvement when using the feature.

Table 3.1: Comparison of Service-Specific Social Features

Feature	Interaction Level	Mode	Difficulty
Chat	Very High	Pair	High
Common Listening	High	Group	Very High
Forums/Wiki	High	Group	Medium
Personal Feed	High	Individual	Low
Song/Playlist Comments	High	Group	Low
Listening Parties	Medium	Group	Very High
Post Embeddings	Medium	Group	Medium
Friends' Listening Activity	Medium	Pair	Low
Following Genres Display	Low	Individual	Medium
Real-Life Events Section	Low	Group	High
Music Wishlist	Low	Individual	Low
External Services Integration	Low	Pair/Group	High

Overall, it is clear that numerous music-related platforms with social features built in are available across the Internet. However, services that focus on the actual audio delivery often support just a subset of the possible features. This, in turn, forces users to use multiple platforms in order to meet their needs. This project aims to bridge that gap, gathering multiple social features in one app.

Based on the research done in the previous and current chapters, the following social features were chosen for implementation, as they provide a good balance between the complexity of implementation and the level of social interaction:

- Chat.
- Personal/Global Feed.[1]
- Song/Playlist Comments.
- Friends' Listening Activity.

These and further requirements for the application are listed in the next chapter **Chapter 4**.

1. In comparison to `last.fm`, a common view with posts from multiple authors is going to be implemented as well.

4 Specification

In this chapter the general outline of the application is given via use cases and functional/non-functional requirements. The individual requirements are constructed based on the 'MoSCoW' criteria [18]. "Must Have" requirements specify functionality without which the application can not be considered finished. Those are the main features and technical aspects that must be implemented. "Should Have" requirements mostly represent features that would greatly improve the user experience, but are not strictly needed for the application to be fully working. "Could Have" requirements are reserved for possible future ideas which most probably would not be implemented in the current scope of the thesis.

4.1 Important Terms

Below is the list of important terms that appear throughout the requirements sections and further chapters:

- **Anonymous User** – A User who is not yet registered in the system or has not logged in.
- **User** – An object containing information such as email, display name and other User parameters.
- **User Profile** – A Frontend-only abstraction which shows Base User information.
- **Artist** - A User with additional privileges, for example, Song or Collection creation.
- **Follower/Followee** - A relationship between a User and another User where one subscribes to another.
- **Friend** - A Followee and Follower who are mutually subscribed to each other.
- **Song** – An object representing a processed audio file with additional metadata such as title or creation date.

- **Collection** – A container that holds multiple Songs.
- **Playlist** – A Collection consisting of pre-existing (already uploaded) Songs.
- **Album** – A Collection consisting of newly uploaded Songs.
- **Playback** – A process during which the User is receiving or manipulating audio data.
- **Playback Item** – A Song or a Collection.
- **Playback Queue** – A User-specific list of Playback Items which determines the playing Song order.
- **Liked Songs** – A private, system-managed Collection that holds the Songs a User has marked as liked.
- **History** – A private, system-managed Collection that records the Songs a User has played.
- **Playback Device** – A device on which the app is currently used.
- **Active Playback Device** – The User's Playback Device that is currently producing audio. All other Playback Devices are passive and mirror the Active Device's Playback position.
- **Friend Activity** – A display of the Songs currently listened to by the User's Friends.
- **Publication** – Textual User input.
- **Attachment** - Non-textual User input. For example, a Song attached to a Publication.
- **Reply** – The same as Publication, but created in relation to another, 'parent' Publication.
- **Feed** - A list of Publications. Either User-specific or Global (from multiple Users).
- **Chat** - A list of Publications specific to two or more Users.

4.2 Use Cases

A use case describes what an actor(User) can do with the system.

While the requirements in the following sections list what the system must do, use cases show the same functionality from the actor's perspective. This can make further modeling simpler, since the minimal subset of actions any given User can take is clearly defined.

The use cases are split into six groups: Account and Identity, Social, Library and Content, Playback, Search, and Communication. Each group has its own use case diagram. The Artist actor inherits from the User actor, so every use case available to a User is also available to an Artist.

The use case diagrams can be found in the `specification/analysis/use_cases/` directory.

4.3 Functional Requirements

Must Have

- The system shall support User and Artist models.
- The system shall allow Anonymous Users to create a User model and log in to the system.
- The system shall allow Users to change model information (name, bio, avatar, etc.) after the creation.
- The system shall support over-the-net audio Playback.
- The system shall provide a way to control the audio Playback.
- The system shall provide a way to display Playback Items uploaded to it.
- The system shall provide each User with a library view of their owned/liked content.
- The system shall limit content presentation based on a specific User. That includes Playback Items, User Profile, User Playback Devices, and other related items.

- The system shall support the creation of Playback Items. Playlists shall be created by all Users. Albums shall be created by Artists only.
- The system shall provide a Playback Queue that the User can modify.
- The system shall allow Users to mark individual Songs and entire Collections as liked.
- The system shall provide Chat capabilities, including direct Chats between two Users and group Chats with multiple members.
- The system shall allow any Publication to be created as a Reply to another Publication.
- The system shall allow Users to add Publications (Comments) under public Collections.
- The system shall provide a personal Feed of a single User's Publications and a global Feed that aggregates Publications across Users.
- The system shall allow Users to follow and unfollow other Users. Mutual follows shall be treated as a Friend relation.
- The system shall have a way to search Playback Items, Users, and Artists.
- The system shall display Friend Activity, that is, the Songs currently being played by the User's Friends.

Should Have

- The system shall support adding Attachments, such as Songs, Collections, or Users, to Publications and Chat messages.
- The system shall provide a notification system.
- The system shall make it possible to filter content by by the viewer's relation to the author.

- The system shall allow a User to the Active Playback Device between them.

Could Have

- The system shall make it possible for multiple Users to listen to Songs in a synchronized manner.
- The system shall provide an Artist interface that allows Artists to see statistics related to their content.
- The system shall have a forum section.
- The system shall provide content recommendations based on a User's listening history

4.4 Non-functional Requirements

Must Have

- The system shall provide access to its contents only to authorized Users. The only exceptions shall be the "Log In" and "Sign Up" pages.
- The system shall make unauthorized access to data as complex as possible, so that even in the case of misconfiguration of access rights, the data are hard to get to.
- The system shall store uploaded content securely.
- The system shall store and transport sensitive User information safely. I.e., it should use safe transport protocols, such as HTTPS and WSS .
- The system should be resistant to the most common web-based attacks: CSRF, XSS, etc.
- The system shall, in case of unexpected failure, remain rendered on the screen and show the appropriate non-technical message to the User.
- The system shall minimize the network load it has on the server and the User device. That includes reducing the data footprint and implementing eager loading strategies.

Should Have

- The system shall provide a responsive search method that reacts dynamically to User input.
- The system shall keep Playback position and the current Song synchronized across a User's open Playback Devices.

Could Have

- The system shall support offline Playback.

- The system should integrate a DRM mechanism.

The next chapter presents the high-level technology choices used for the implementation.

5 Implementation Planning

Before the actual implementation could begin, it was necessary to choose the languages, frameworks, and other technologies that would shape the overall architecture of the application. As mentioned before, the goal of the thesis is a web application, hence, this chapter focuses on the technologies mostly suitable for web applications. This chapter discusses the following areas: Audio Processing and Playback, Authentication, Server–Client Communication, Backend, and Frontend.

5.1 Audio Processing and Playback

Audio Processing and Playback are the most important aspects of the application and were researched first. There are four main approaches used today: Basic Download, Progressive Download, Streaming, and Peer-to-Peer.

- **Basic Download**
  Basic Download works by downloading the entire file before Playback begins. It does so by making a standard HTTP GET request to which the server sends back an HTTP response with the media as raw binary data in its body. [19] Only when the full transfer has completed, the user can start the Playback.

  This method has several limitations: for high-quality files, it typically requires significant storage space and time to download. Additionally, if the user's connection is unstable, it can make the application (which relies on playback) unusable. In cases of poor network conditions, the situation can be partially improved by providing multiple representations of the same file, such as a highly compressed version alongside a lossless one. However, after the download has started, the system is unable to adapt to the changing network state. In other words, the user is tied to the version of the file that has started to download and cannot benefit from, for instance, switching from a slow mobile network to a fast Wi-Fi connection.

- **Progressive Download** [1]
  Progressive Download also utilizes HTTP to get the content, but the client does not need to download the entire file before playback starts. Instead, it fetches a short initial part, begins playback, and continues downloading further parts as needed.

  This is usually implemented with HTTP byte-range requests. [21] To make this possible, the client relies on audio container metadata (e.g., the header of an MP4 file) that maps playback time to byte offsets. [22] With that mapping, the player can request the next few seconds of audio and also seek by requesting a different range.

  In comparison to Basic Download, since the file no longer needs to be downloaded fully, storage becomes less of an issue. The player only needs a small buffer in memory, and older data can be discarded once it has been played. The browser may still cache recently fetched ranges, but the application does not need to persist complete files on disk.

  In addition, it becomes possible to approximate adaptive playback. One approach, presented by Färber Nikolaus [22], involves storing multiple representations of the same audio file as separate tracks inside a single container file (e.g., MP4) and switching between them during playback. The client then continues requesting parts of the file using HTTP byte ranges, but chooses the track that best matches the current network conditions. However, seamless switching requires the representations to be prepared in a compatible way: for example, corresponding parts of each track must cover the same playback time intervals and start at safe switching points.

- **Segmented HTTP Streaming**
  With Segmented HTTP Streaming, the audio file is first split into many small audio files (chunks), and a special manifest file is then generated that lists the chunks and describes which time range each chunk covers (usually a couple of seconds). [23] The most common implementations of this approach are MPEG-DASH and HLS, both of which are described in **Chapter 6**. The

1. Sometimes called 'Pseudo-Streaming'. [20]

client then downloads the individual chunks and uses them for playback instead of working with byte range requests for a single large file, as was the case with Progressive Download. Since the manifest provides a mapping of chunks to audio time segments, this makes it possible to play the audio from any point and enables seeking.

Moreover, the original audio file can first be encoded into multiple formats of different sizes. Then, it is possible to preprocess those files and create a single manifest that allows choosing chunks of different sizes and quality for the same time slot. As a result, chunks can be selected based on network conditions or device capabilities, providing a more optimized playback.

Compared to Progressive Download, Streaming makes adaptive playback simpler, because the manifest provides a standardized way to describe multiple representations and their timing. It also simplifies retries and caching, since each chunk is fetched as a separate HTTP resource and can be handled efficiently by standard HTTP caches and CDNs. [24] Finally, the client logic becomes more straightforward: instead of computing byte ranges inside a large file, the player selects the next chunk from the manifest.

- **Peer-to-Peer**
  In standard P2P audio transfer, audio data is shared directly between users' devices rather than being served from a central server. [25] Each user acts as both a client and a server, sending data to and consuming audio from other users in the peer-to-peer network. This method can reduce server load and improve access speed, but it relies heavily on user participation and network conditions.

Streaming was chosen as the playback method for this thesis.

Basic Download was excluded because it requires downloading the whole file before playback can start, making the usability of the application limited. Progressive Download improves startup time and seeking, but adapting to changing network conditions is not standardized and would require custom preprocessing of audio files on the

server side, as well as writing client logic that could utilize this approach. Peer-to-Peer transfer was also excluded, since it would rely on users' devices as sources of audio data, making playback speed dependent on peer availability and location.

Segmented HTTP Streaming addresses these issues by defining adaptive playback directly in the manifest and delivering the audio as small chunk files. Additionally, its integration into the service, as will be demonstrated later in **Chapter 6**, is straightforward.

The next key choice would be the authentication method, as it can significantly influence both the design of the Frontend and Backend.

5.2 Authentication

There are many different methods of authentication available, but only those suitable for web applications are considered: Basic Authentication, Session-Based Authentication, Token Authentication, and Third-Party Authentication (OAuth2).

- **Basic Authentication**
  Basic Authentication sends a username and password with each HTTP request made from the browser. They are placed in the HTTP `Authorization` header, typically Base64-encoded. [26]

  The browser then usually caches those credentials after the first successful login and automatically attaches the header to subsequent requests. [2]

  Nonetheless, it has multiple problems, even if used over HTTPS. The value sent in the `Authorization` header is the User's actual password, Base64-encoded for transport but not encrypted in any way. If the browser side is compromised, or if this header ends up in a log or telemetry record, for instance after HTTPS is terminated at an internal proxy [29], what leaks is the password itself rather than some opaque value. Revocation, therefore, requires a password change, which can not be transparently done without User interaction.

2. This is usually not clearly stated in the browser documentation. However, for example, in the case of Mozilla this is implied in bug tracker tickets and user documentation [27] [28].

Moreover, there is no protocol-level notion of *logout* or per-device sessions. Since the browser keeps attaching the cached credentials on its own, the Frontend has to resort to workarounds to sign a User out, such as sending a request with deliberately wrong credentials in the `Authorization` header so the browser discards the cached value. Otherwise, the User has to clear the browser cache manually [27].

- **Cookie-Based Session Authentication**
  Session authentication makes it possible to identify users without sending their credentials on every request. Initially, the user must log in with their credentials in the same manner as for Basic Authentication. But, instead of the browser caching and reusing the credentials, the server that processed the request creates a *session* record, usually a data structure holding relevant user information (ID, roles, etc.), and instructs the browser to set a cookie with the session ID. This ID is later sent on subsequent requests instead of the user credentials and is used to identify the logged-in user. [30]

  Compared to Basic Authentication, this enables server-side logout and revocation (by deleting or invalidating sessions), per-session security policies such as device tracking, inactivity timeouts, centralized control of active logins, and a simpler implementation of features that depend on temporary state.

  The main trade-off is the need for a session store, typically implemented using a relational database or an in-memory key–value store. This introduces additional operational complexity, infrastructure dependencies, and potential bottlenecks if not configured correctly.

- **Token Authentication**
  Token Authentication replaces raw credentials with a token that the client presents on each request instead of the login and password. The token is issued by the server after a successful login and is later sent by the client in the `Authorization` header, usually under the `Bearer` identifier.

  There are two common variants of this approach. The first one is the *database-backed token*: the server stores the token together

with the corresponding user record and looks it up on every request. This is similar to a session, but the identifier travels in a header instead of a cookie. Such tokens can be revoked at any moment by deleting the stored record. The second variant is the *self-contained token*, such as JSON Web Tokens (JWT). Here the token itself carries a payload with the user identity and permissions, so the server only needs to verify the signature with a shared secret or a public key, without any lookup. This removes the dependency on a central token store and can simplify scaling, since any instance that knows the verification key can accept requests.

Token Authentication works well for clients that are not browsers, such as command-line tools, mobile applications, and IoT devices, where sending cookies is inconvenient and the token can be securely stored. It is also commonly used for public APIs and distributed systems, where independent services need to authenticate to each other without sharing a central session store.

Nevertheless, Token Authentication has a few notable tradeoffs. For self-contained tokens, revocation is hard: once issued, a token usually stays valid until its expiry time has elapsed. It is possible to keep a *revocation list* and check every request against it, but that brings back state, coordination, and additional latency. In a single-page application, the token also has to be stored somewhere on the client side. If held in `localStorage`, it is reachable from any script on the page, so a Cross-Site Scripting (XSS) injection can obtain it, while session cookies marked `HttpOnly` are not exposed to scripts. Moreover, the Frontend has to attach the token to every request and handle token refresh explicitly, whereas cookies are sent by the browser automatically.

- **Third-Party Authentication (OAuth2)**
  Third-Party Authentication delegates user login to a third-party identity provider (via the OAuth 2.0 protocol), such as Google, GitHub, and others. [31] Instead of handling passwords di-

rectly, the application trusts this provider to authenticate the user and assert their identity.

In the standard authentication flow, the application redirects the user to the provider's endpoint, where the user logs in. The provider then redirects back with an authorization code, which the application exchanges for tokens that can be later used to obtain user information. Using that information, the application can then create internal user models with the provided identity.

Thus, this approach offloads password storage, multi-factor authentication, and account recovery to the provider. However, it should not be the only authentication method the application provides, since it requires the user to have an account with one of the providers.

Based on the characteristics of the authentication methods, it was decided to use the Session Authentication.

Compared to Basic Authentication, it avoids sending user credentials with every request, supports explicit logout, per-session security policies, and extra session-related data. Compared to Token Authentication, sessions offer simpler revocation, do not require managing tokens, and are sufficient for a web application that does not need stateless or cross-service authentication. Third-Party Authentication based on OAuth 2.0 can be added on top of this design.

Another important concept that must be addressed is the server-client communication.

5.3 Server-Client Communication

Some of the features outlined in **Chapter 4** (such as playback handling, and immediate content updates) require bi-directional communication. Hence, a method for communication from the server to the client must be added.

Because this is a web application, techniques such as Webhooks, gRPC, Pub/Sub, and others that are not directly supported by browsers are not discussed. Traditional Polling is also not considered due to its performance limitations and inefficiency. P2P techniques (e.g., We-

bRTC) are not listed because the application's architecture is client-server, not peer-to-peer.

This thesis focuses on four commonly used methods for server-client communication: Long Polling, Server-Sent Events, WebSockets, and Push API.

- **Long Polling**
  Long Polling is a technique where the client sends an HTTP request and waits until the server responds, holding the HTTP connection open until a response is received (or a timeout occurs). Once a response is received, the client immediately issues another request, therefore listening to server messages continuously. [32] While this method works in environments where server-to-client messages are rare, it is unsuitable for high-throughput scenarios (especially for frequent and small payloads), as it introduces a significant overhead by repeatedly opening and closing HTTP connections. [33]

- **Server-Sent Events (SSE)**
  SSE is a one-way communication protocol that allows the server to send data to the client over a single, long-lived HTTP connection.

  Firstly, the client establishes the connection via an HTTP request, to which the server answers with a response that has its *content type* set to `text/event-stream`. The server can then reuse that open connection to send an unlimited number of responses (events). [34] Compared to Long Polling, SSE does not need to open a new HTTP connection for every update, reducing the number of requests. At the same time, it is still based on plain HTTP, so it works in most browsers and fits well into networking setups.

  However, this protocol also requires additional connection handling logic: the proxy and the server must keep track of all connected clients. Moreover, special handling on the Backend must be added in order to properly route messages. [35]

- **WebSockets**
  WebSockets, in comparison to SSE, provide full-duplex communication over a single TCP connection. This is achieved by

firstly creating a standard HTTP connection and then upgrading it to the WebSocket protocol via the WebSocket Protocol Handshake. [36] This method allows messages to be sent both ways at any moment and is suited for applications that require frequent two-way communication. [33]

However, adding WebSockets to an application, as well as SSE, requires additional infrastructure and logic for managing connections and message routing. In addition, WebSocket messages do not follow the normal HTTP request-response model. They cannot be cached by standard HTTP caches [37] or CDNs, and they do not benefit from built-in features such as idempotent retries.

- **Push API**
  The Push API is a specialized web interface that allows a server to send messages to the browser without a preceding request from the client. [38] After the user grants permission, the browser creates a *push subscription* and exposes a special endpoint. [39] The server can then send data to that endpoint without additional negotiation. When such a request is received, the browser validates it and queues the message for processing by a special background script, called a *service worker*. [40] Each domain (website) can register its own service worker, which is typically dormant and is only started when events such as push notifications need to be handled.

  While this mechanism is useful for notification systems, where short delivery time is not critical, it is unsuitable for interactions that require an immediate response, such as updating the User's current Playback state. Messages can be delayed inside the browser because multiple sites may have pending push messages to process, and the browser can throttle service worker execution and limit the resources available to it. As a result, push notifications typically have higher and less predictable latency compared to direct communication protocols, such as WebSockets. [38]

Based on the requirements outlined in **Chapter 4**, at least three features would require server-to-client communication:

- **Playback state synchronization.** Any Playback Device the User acts on has to send the change to the server, whether it is a play, pause, seek, or song switch, and the Active Playback Device additionally sends periodic position updates to correct drift. Even at a moderate rate, these events add up, and each one would otherwise need its own HTTP request. The server then forwards the new Playback state to the User's other Playback Devices to keep them in sync.

- **Real-time typing indicators in Chats and Comment threads.** The client would send multiple short updates to the server while the User is typing. The server would then broadcasst these events to the other participants, so that the current typing activity is displayed in real-time.

- **Live Chat messages and Comments.** When another User sends a message in an open Chat, or posts a Comment under a Collection that the User is currently viewing, the new content should appear on the screen immediately, without periodic polling from the client.

The initial implementation used SSE. The read-path features in the list above (Live Chat messages, Comments, typing indicators) are mostly server-to-client and would have been covered by a long-lived SSE stream for the receive path with ordinary HTTP requests for the send path. Playback synchronization, however, does not fit this split well. When the User drags the position slider quickly, the primary Playback Device sends a short burst of position updates to the Backend, which then forwards them to the secondary Devices. With SSE for the receive path and one HTTP POST per update on the send path, every update in that burst is a separate HTTP request. Even with a single User this adds extra lag, and with many Users active at the same time it puts real load on the server.

Additionally, mature WebSocket support is widely available in modern Python web frameworks, often as a built-in feature with primitives such as named groups and per-user broadcast. Equivalent SSE support is less standardized and typically requires a separate pub/sub layer.

For these reasons, WebSockets are used as the primary mechanism for real-time communication. The drawbacks listed above are accepted in exchange for a uniform bidirectional channel and lower integration cost with the chosen Backend framework.

These choices define what the Backend and Frontend have to support, and Backend is discussed next.

5.4 Backend

5.4.1 Language

Python was chosen as the language for the Backend of the application. It is well-suited for web development, since it provides many libraries and built-in interfaces for HTTP APIs, database access, background processing and other areas which are going to be used in the implementation. TODO (also that its interpreted and not strongly typed) Its syntax also allows fast prototyping, which is helpful while iterating on the data model and the overall app architecture. Performance is good enough for the scale of this project, as the application is not expected to handle very high request rates, and the most computationally heavy task, audio processing, is handed off to external CLI tools (see **Chapter 6**).

5.4.2 Python Framework Comparison

There are currently three widely used frameworks available for Python: FastAPI, Django, and Flask.

- **FastAPI**
  As the official website states:

  > 'FastAPI is a modern, fast (high-performance), web framework for building APIs with Python, based on standard Python type hints.' [41].

  Its main emphasis is compatibility with Python's `asyncio`, which makes it much easier to write asynchronous code in comparison to the other two frameworks. In addition, it integrates well with

Pydantic [42] making user input validation straightforward. Moreover, routing/middleware is trivial to configure. [43]

However, FastAPI is relatively new and simple. It can work well for small or highly custom systems, but for standard setups, it lacks many convenience wrappers. For example, the WebSockets integration does not provide connection lifecycle management or message broadcast capabilities. [44] Additionally, no predefined templates are available for basic CRUD operations; therefore, they must be implemented manually, which adds unnecessary complexity.

And lastly, despite the steady growth of FastAPI usage, the community support is still not as wide as for Django or Flask. [45]

- **Django**
  Django is a "batteries-included" [46] framework in the sense that it has tools for most of the common web development tasks. It provides built-in user session management, easy CSRF, CORS and XSS protection setup, built-in file storage abstractions, and integrated handling of media files (images, audio, etc.). Support for REST APIs can be added through `django-rest-framework` [47], which provides generic views, serializers, and routers that make the implementation of CRUD endpoints quick with minimal custom code on top. Real-time functionality can be implemented using `django-channels` [35], an official Django package that adds connection lifecycle management, group-based message broadcasting, and integration with Django's authentication and session systems. As the documentation states, one of the main advantages of this framework is the time savings it offers. [48]

  Nonetheless, some notable drawbacks include its opinionated design choices, which can make it hard to add custom logic. Furthermore, the size of the resulting codebase can be larger because the framework ships with many modules that may remain unused, while still occupying storage space (for example, RSS integration or server-side rendering modules).

  Finally, the most significant disadvantage is its limited support for asynchronous processing [49]: plenty of ORM operations

(e.g., transactions), database connection management, and many third-party middlewares/libraries remain synchronous / async-unsafe.

- **Flask**
  Flask is also a lightweight Python web framework. [50] It provides only a small core with routing, request/response handling, and templating, while most other functionality is added through extensions. This modular design is suitable mainly for small services or prototypes with a limited and clearly defined scope.

  However, for typical JSON-based APIs, FastAPI offers similar functionality with fewer dependencies and less setup, while Flask usually requires several additional libraries and manual configuration to achieve the same result. At the same time, Flask, like Django, is based on the synchronous WSGI interface. WebSocket support is typically added via additional tools, such as SocketIO [51], which requires the Frontend to use the same library instead of the standard WebSocket API, introducing unnecessary coupling. As a result, Flask effectively combines the disadvantages of the other two frameworks: it lacks Django's integrated ecosystem and does not provide FastAPI's modern asynchronous features. In the context of this application, it does not bring any clear advantage over Django or FastAPI.

Summing up, of the three frameworks, Django is the most suitable one. For a music streaming platform, its media handling capabilities, WebSocket support via `django-channels`, and CRUD endpoint abstractions of `django-rest-framework` lead to a better developer experience, shortening the overall implementation time. The limited support for asynchronous code is also less of a problem in practice for this project, since WebSocket handling already requires Django to run under ASGI through `daphne` (which `django-channels` depends on). For the synchronous request paths, Instagram has shown that Django can still handle large loads. [52]

Having established the Backend framework, the next step was to choose the database.

5.4.3 Database

The Django framework supports several relational databases: SQLite, PostgreSQL, MySQL/MariaDB, and Oracle. Oracle is excluded due to its commercial license [53]. Otherwise, for a project of this size, any of these options would be fast enough, because the expected number of users and the amount of stored data are not significant. Therefore, performance is not the primary factor in choosing a database.

Instead, the choice is based on the available features and how well they are integrated with Django. In this application, two requirements are especially important: full-text search and UUID primary keys (see **Chapter 6** for details). And for these requirements, PostgreSQL is the most suitable option.

First, PostgreSQL offers good full-text search support and additional tools for fuzzy matching, such as trigram indexes. [54] As a result, it can be used as the base for full-text search instead of a separate service like Elastic or Sphinx. [55, 56] Django exposes these features through the `django.contrib.postgres` module, which simplifies the creation of full-text and trigram-based queries directly from Python code. [57, 58]

Second, PostgreSQL has native support for UUID values via a dedicated `uuid` column type. [59] This type stores UUIDs in a compact binary form and allows efficient indexing and comparison. [3] In contrast, in many other databases UUIDs must be stored as plain-text (e.g., `CHAR(36)`), since they do not support this format natively. This is less space- and index-efficient. Django's `UUIDField` also maps cleanly to PostgreSQL's native type, so UUID-based identifiers can be used without extra configuration.

Lastly, with the database chosen, the Frontend application type, language, and framework had to be selected.

5.5 Frontend

5.5.1 Application Type

Modern web applications are typically built either around server-side rendering (SSR), where most of the HTML is generated on the

3. If using time-based UUID types, e.g., UUIDv7 [60]

server and then retrieved by the browser to be later rendered, or around client-side rendering (CSR), where most HTML is generated in the browser using JavaScript. A single-page application (SPA) is an application that keeps a single HTML document loaded and updates the user interface dynamically on the client, usually relying on CSR, sometimes combined with SSR for the initial page load.

In an SSR setup, frameworks such as Django generate most of the user interface on the server and send complete HTML pages (or partial updates) to the browser, often only requiring small pieces of JavaScript for interaction. [61] In contrast, an SPA typically fetches JSON or other data from the server and, based on it, renders and updates the user interface directly in the browser. [62]

The research for the Frontend began by deciding which of these approaches to use. For many web applications, rendering HTML on the server reduces client-side complexity, avoids duplicating logic between browser and Backend, and works reliably across a wide range of devices and browsers, making it a simpler and more robust default choice.

However, in this project, the application must maintain a lot of state directly in the browser: the current playback position, the playback queue, the volume level, the chat state, etc. These values change frequently and in small steps, sometimes several times per second, and require immediate feedback for the user. Moreover, the user interface must respond to events sent by the server over WebSockets, such as playback updates or chat messages. A purely server-rendered approach with full page reloads or round-trips for each small change would be unsuitable for this kind of highly interactive, real-time interface and would complicate state handling across devices. Because of these requirements, the Frontend is implemented as a client-side rendered single-page application (SPA).

5.5.2 Language

Currently, there are many libraries and frameworks in different languages that can be used to write web interfaces. For example, it is possible to create them even in low-level languages, such as C [63] or Rust [64]. Nevertheless, JavaScript remains the industry standard because it is still the only general-purpose language directly supported

by browsers. It means that JavaScript is the only language that can be executed natively by the browser's runtime, without the need for previous pre-processing (e.g., compilation to WebAssembly). This simplifies the development, build, and deployment process, and for this reason the client application is based on JavaScript.

The project, however, does not use plain JavaScript, but rather TypeScript on top of it. TypeScript is a typed superset of JavaScript that compiles to plain JavaScript, so the code that the browser actually runs is still JavaScript. The benefit comes from the additional type checking performed during the build step, which catches many simple errors before the code is run.

5.5.3 Framework

Modern Frontend development is mostly based on libraries or frameworks such as React, Vue, or Angular. [65] For this project, React [66] was chosen.

React is a library for building user interfaces from reusable components. Its core only handles rendering, so common needs such as routing, state management, data fetching, and UI components are added through mature ecosystem libraries instead of being written from scratch.

React provides a built-in mechanism for component state: when a value in that state changes, the parts of the interface that depend on it re-render automatically. This matches the requirements established earlier in this section, since values such as the Playback position, queue, volume, and live Chat messages change frequently and need to appear on screen immediately, without manual DOM updates.

Components are written in JSX, an extension to JavaScript that allows HTML-like markup to live next to the component code. This makes it natural to define repeating interface pieces once, such as a Song row, a Chat message, or a Comment, and reuse them throughout the application.

React DevTools [67] also makes it possible to inspect the currently rendered components and their state directly in the browser, which simplifies debugging. React is widely adopted [68], so examples and solutions to common problems are easy to find.

5.6 Summary

The platform follows the Model–View–Controller architectural pattern [69] and is organized into three logical layers:

- **Model**
  The Model layer is responsible for representing and manipulating the application data. In this project, it consists mainly of the Django models with entities such as Users, Songs and Collections; and the data processing logic, for example audio conversion or data querying.

- **View**
  The View layer is what the user interacts with directly. It presents the data from the Model layer and turns user actions into requests that can be processed by the rest of the system. Here, the View is implemented as a single-page application in JavaScript using React. It renders the interface, displays information such as playback state or friends' activity, and sends user actions (for example, play/pause commands or chat messages) to the Controller.

- **Controller**
  The Controller layer connects the View with the Model. It receives HTTP and WebSocket requests from the Frontend, applies cookie-based session authentication and access control, and then calls the appropriate Model logic. In this platform, the Controller corresponds to the Django REST API and WebSocket endpoints.

In other words, the Django models and database layer form the Model, the React single-page application acts as the View, and the Django-based REST and WebSocket endpoints play the role of the Controller that coordinates communication between them.

The next chapter, **Chapter 6**, describes the concrete implementation of these parts and how they work together to form the complete music streaming platform.

6 Implementation

This chapter presents the created platform: first it introduces the configuration of the Backend and the Frontend, and then the individual parts of the system, such as User Management and Representation, Audio Processing, Publication Creation, etc.

In contrary to the **Chapter 5**, this chapter will not split Backend and Frontend into different sections, but rather will discuss the complete system components. In other words, each part will contain the data model, the business logic, the API and the Frontend components used for the presentation and interaction.

A high-level overview of the entire system can be seen in the following diagram 6.1.

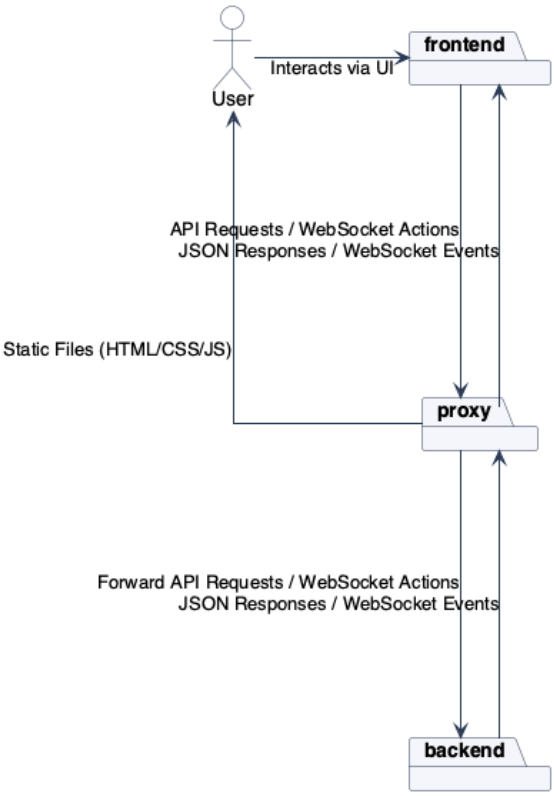

Figure 6.1: System Overview

The high-level Backend architecture, with its main parts and the connections between them, is shown in 6.2. The corresponding Frontend architecture is presented in 6.3. The per-app Backend class diagrams, the Frontend component hierarchy can be found in the `specification/structure` directory.

6.1 Configuration

This section lists the concrete frameworks, libraries and tools that were used when creating the system.

6.1.1 Backend Configuration

- Language: `python@3.14`
- Package Manager: `uv` [70]
- Framework: `django@5.2`
- WebSocket Management: `django-channels`
- ASGI Server: `daphne` [71]
- API: `django-rest-framework`, `django-filter` [72],
- Backgroud Tasks: `celery`
- Cache/Key-Value Store: `redis` [73]
- Authentication: `dj-rest-auth` [74]
- Database: `postgres:17`

6.1.2 Audio Processing Configuration

- Transcoding: `ffmpeg@8.0.1`
- Packaging: `shaka-packager@3.4.2`

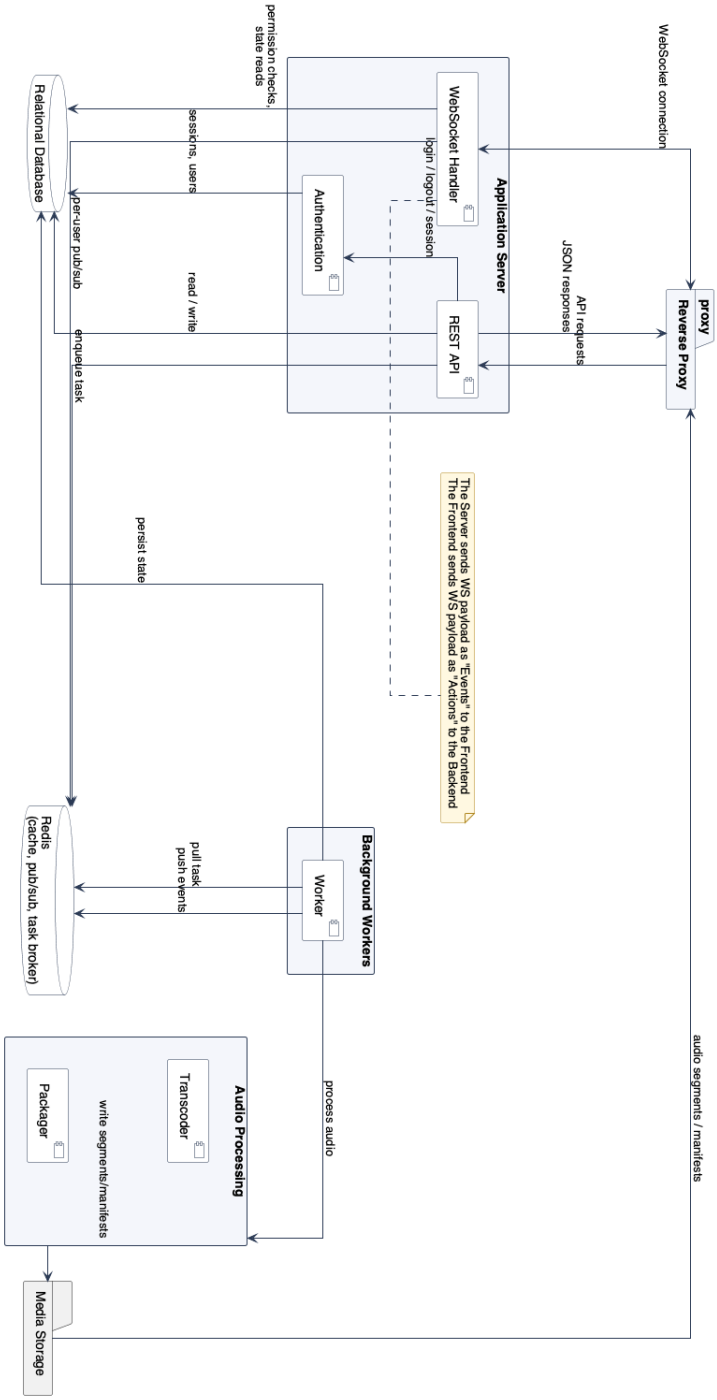

Figure 6.2: Backend Architecture

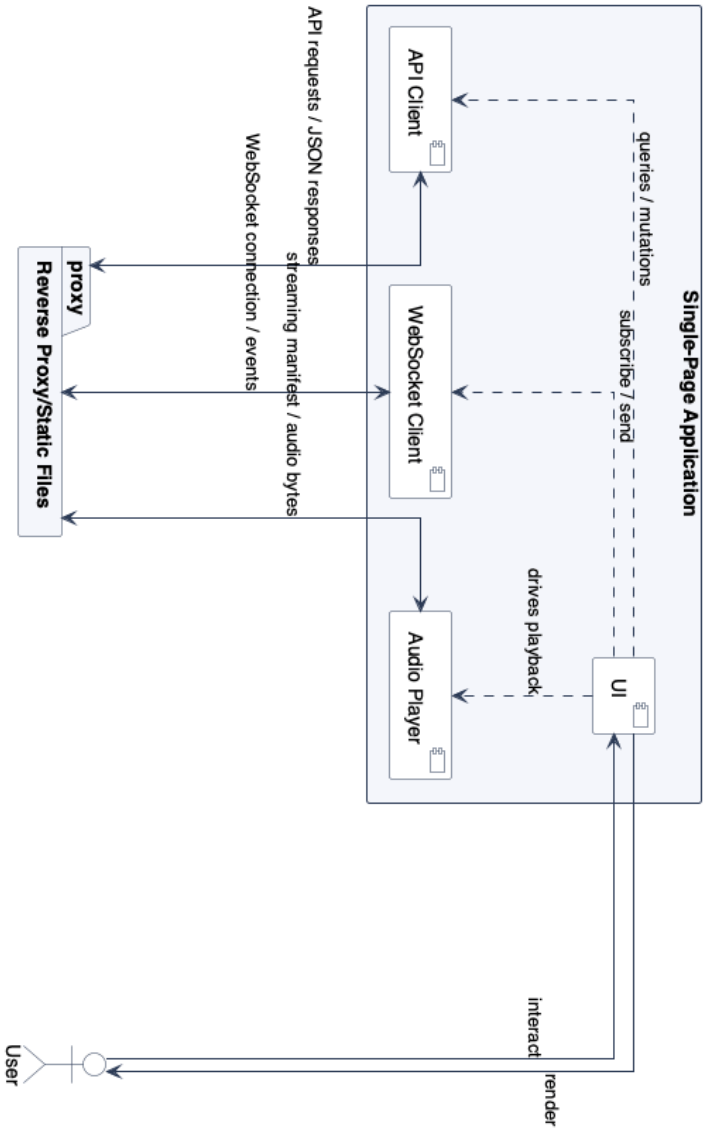

Figure 6.3: Frontend Architecture

6.1.3 Frontend Configuration

- Language: `typescript@5.9.3`
- Package Manager: `npm`
- UI: `react@19.2.0`
- Bundler, Dev Server: `vite`
- Styling: `tailwindcss`, `shadcn/ui`, `@radix-ui/*`
- Routing: `react-router-dom`
- Server state + HTTP: `@tanstack/react-query`, `axios`
- Forms, Data Validation: `react-hook-form`, `zod`
- Audio Playback: `shaka-player`

6.1.4 Deployment Configuration

- Docker
- Docker Compose
- Reverse Proxy: Caddy
- VPS Hosting: Hetzner
- Mail Service: Mailgun
- Domain Management: PorkBun

6.2 Audio Processing and Streaming

This section describes how an uploaded audio file is turned into output that can be streamed to the Frontend. The Streaming Protocols are described first, since they constrain the codec and container choices made by every later stage of the pipeline.

The chapter stays at the level of what each stage does and what parameters it uses. The deeper rationale (protocol trade-offs, codec

comparisons, CBR versus VBR, sample-rate theory, loudness normalization explanation, and the per-flag breakdown of every CLI invocation) is presented in `text/extras/audio_processing_details.tex`.

6.2.1 Streaming Protocols

Two protocols are used: MPEG-DASH [75] on devices that support Media Source Extensions, and HLS [76] on iOS, which does not support MSE. The basic principles behind both protocols were covered in the Segmented HTTP Streaming section.

Both protocols expect audio split into short chunks paired with a manifest. The Packaging step is the one that produces those.

6.2.2 Packaging

Packaging cuts the transcoded audio into short fragmented MP4 segments and generates a DASH manifest (`*.mpd`) and an HLS master playlist (`*.m3u8`) that point at them. [77] The segments follow Common Media Application Format (CMAF), so a single set of chunks works for both protocols. The output is one initialization segment (`*_init.mp4`) per representation and a sequence of media segments (`*.m4s`).

Packaging only arranges existing audio. The CLI tools that produce and analyze that audio in the first place are introduced next.

6.2.3 Processing Tools

Three CLI tools drive the pipeline: FFmpeg [78] for Transcoding and audio analysis, FFprobe (shipped with FFmpeg) for reading source metadata, and Shaka Packager [79] for the Packaging step. FFmpeg can package too, but its output had compatibility issues with Shaka Player [80], used on the Frontend for Playback, so a separate packager was chosen.

With FFmpeg in place as the encoder and analyzer, the Transcoding configuration it runs can be described.

6.2.4 Transcoding

Transcoding decodes the source audio and re-encodes it into the target codecs and bitrates. Raw audio formats such as WAV are too large for Streaming, for example a 3-minute stereo 24-bit 44.1 kHz recording is roughly 50 MB. Lossless FLAC and lossy AAC/HE-AACv2 were selected as the output codecs, since HLS only allows a small set of audio codecs. [81] The AAC encoder used is `libfdk_aac`, chosen over the other AAC encoders shipped with FFmpeg for its quality and licensing terms.

The output configuration was chosen based on existing research: [82, 83, 84, 85, 81, 86]

- **AAC**: 96, 160, and 320 kbps
- **AAC-HEv2**: 24 kbps
- **FLAC**: lossless

Multiple bitrates support adaptive streaming. The AAC tiers use Constant Bitrate (CBR), so the Frontend's Adaptive Bitrate logic has a stable per-tier bandwidth to pick from.

Transcoding assumes a clean, well-bounded input. The Audio Preprocessing step described next ensures that each upload is valid and within the proper limits, and also captures the metadata of the result.

6.2.5 Audio Preprocessing

Before Transcoding, the upload is validated and normalized. After Transcoding, the loudness is measured. The three sub-blocks below cover input validation, metadata extraction, and the normalization/loudness filter chain.

Input validation

The upload passes through a chain of checks. The file-size and MIME checks run on the raw bytes. Other checks use values reported by FFprobe. Together they restrict the worst-case storage and Transcoding time of a single upload:

- **MIME type**, inspected with `python-magic` [87], must be `audio/*` and belong to an allow-list of supported subtypes.
- **Codec**, reported by FFprobe, must belong to an allow-list of well-tested codecs.
- **File size**: up to 2 GB, roughly a 2-hour 24-bit, 48 kHz stereo WAV.
- **Duration**: up to 7200 s (2 h).
- **Sample rate**: 1 Hz to 384 kHz.
- **Channel count**: 1 to 8.

The allow-lists are defined in `streaming/audio/config.py`.

Input metadata extraction

FFprobe reads the source codec, sample rate, channel count, bit depth, duration, and bitrate. These values are reused by the input validation above. They also narrow which outputs the Transcoding step produces: FLAC is skipped for lossy sources, and AAC tiers whose target bitrate is above the source bitrate are dropped. The exact invocation and per-flag breakdown is in `text/extras/audio_processing_details.tex`.

Normalization and loudness

Three normalization filters run as a single FFmpeg `-af` chain applied inside every Transcoding subprocess. A separate loudness measurement then runs after Transcoding. The steps are:

- **Channel normalization**. Sources with three or more channels are downmixed to stereo. Mono and stereo inputs are kept as-is.
- **Sample rate normalization**. Each source is resampled to the base of its rate family (44.1 kHz or 48 kHz). Files below 44.1 kHz are kept as-is. Non-family rates default to 44.1 kHz.
- **Bit depth conversion**. All audio is converted to signed 16-bit samples, since `libfdk_aac` does not accept higher precision.

- **Loudness measurement**. After Transcoding, FFmpeg's `ebur128` filter produces an integrated value in Loudness Units relative to Full Scale (LUFS) and a true peak in dBTP. The measurement runs on the FLAC output if it was produced, otherwise on the first AAC output. Both values are stored on the Song and used by the Frontend player to apply a consistent Playback gain.

The concrete filter strings and the ebur128 invocation are listed in `text/extras/audio_processing_details.tex`.

Once Transcoding and Packaging finish, the original upload and the intermediate transcoded MP4s are removed. Only the packaged segments, manifests, and the loudness values stored on the Song row survive.

The Backend code that drives all of these stages is described next.

6.2.6 Backend Wrappers

The Backend code lives under `src/backend/musikk/streaming/audio/` and the class layout is shown in `text/diagrams/AudioClassDiagram.png`. The exact CLI invocations the wrappers emit, with a per-flag breakdown, are in `text/extras/audio_processing_details.tex`.

Pre-pipeline validation (the file-size, MIME, and FFprobe-based checks from the previous subsection) runs in `validators.py` before any audio subprocess is started. The pipeline itself, `AudioProcessingPipeline` in `processing_pipeline.py`, threads a shared `ProcessingContext` through a chain of `Step` instances. The configured chain runs `FFmpegStep`, then `LoudnessMeasurementStep`, then `ShakaPackagerStep`. Each step has a `rollback` method. If a step raises, the previously executed steps are rolled back in reverse order. The rollback clears the per-run temporary directory and removes any output already written to Django storage.

`FFmpegStep` runs one FFmpeg subprocess per output tier in parallel via a `ThreadPoolExecutor` sized to the number of selected tiers. Threads are used rather than processes because FFmpeg itself is multithreaded inside each subprocess, so the Python side only waits on the subprocess to finish. A process pool would add overhead without speeding anything up.

How this pipeline is launched as a background task, and how its status reaches the Frontend, is covered later in the Audio Upload section.

With the Backend producing CMAF segments and both manifests in Django storage, the Frontend only needs a player that can consume either manifest format. That player is described next.

6.2.7 Frontend Playback

On the Frontend the Playback is achieved via `shaka-player`. The individual components can be found under the ... dir

6.3 WebSocket Communication

This section describes the WebSocket setup, i.e., how the WS channels are created, and how the server or the client sends/receives the WS messages.

Messages that are sent from the server to the client are internally called "events", while messages from the client to the server - "actions".

6.3.1 Backend Handling

The Backend WS configuration can be found under the `src/backend/musikk/websockets` directory.

The `BaseConsumer` class is responsible for processing the incoming WS connection attempts: it first validates that the User is authenticated, adds it to a broadcast group under the `user_*user uuid*` key, and creates any relevant event processing classes (see further sections for more information). In addition, it is responsible for sending the "events" and routing the received "actions" to their handlers.

A `/ws/user` endpoint is exposed and is used for connection establishment.

6.3.2 Frontend Handling

Frontend WS configuration is located in the `src/frontend/musikk/src/ws` directory.

The `WSClient` class provides methods for connection creation/closure, "action" dispatch, and "event" processing.

Since many different parts of the Frontend may want to react to the same "events", it also exposes a Pub-Sub mechanism that lets other functions/components register custom callbacks in reaction to "events".

In addition, a basic reconnect mechanism is added, so that the connections are not dropped indefinitely, for example, during short network outages. Moreover, a queue is added for outgoing "actions", so that they are not lost, if the client is not able to send them at the time of creation. All created "actions" are first added to the `pendingMessages` list, and only if the connection is established and healthy are then sent to the server.

A separate `WebSocketManager` class manages the WS connection lifecycle. It monitors the amount of open connections and provides an interface to create/delete them. It was mostly added due to the duplicate connection issues which appeared when using React in development mode.

With WebSocket Communication discussed, User-related functionality is presented next.

6.4 User Management

This section outlines the User data model and presentation layers.

The relevant Backend code is located in the `src/backend/musikk/users` directory, the Frontend code in the `src/frontend/musikk/src/features/user` directory.

6.4.1 User Data Model

Users in the system are represented by a `BaseUser` model which holds a discriminator `role` field that can be of two types: an "Artist" or a "Base". An "Artist" role gives the User additional permissions, such as uploading new Songs or creating Albums (see ...).

Each user is uniquely identified by their email. The `display_name` defaults to a random string and does not need to be unique.

Apart from basic configuration, Users can also have relations to other Users, which are modeled by the `UserFollow` model. The system intentially does not create a separate model for Friend relations: this is inferred by a presence of a "mutual follow", i.e., two existing `UserFollow` relations where the `from_user` and `to_user` foreign keys are mirrored in each model instance.

6.4.2 API

The API provides a standard set of endpoints: a `MeView` which allows the User to retrieve or update their information, a `UserRetrieveView` that returns the non-sensitive information about any specfic User in the system; a `FollowedView` which lets Users follow/unfollow other Users and retrieve the Users they Follow. `FollowersView` and `FriendsView` return the Followers of a User and their Friends respectively.

The endpoints access is gated by permissions so that, e.g., only User's Friends would be able to see their Followers (defined in `src/backend/musikk/users/`

6.4.3 Frontend Presentation

On the Frontend each User has a `ProfilePage` which is used to view and edit User's information. When visiting another User's Profile it is possible to follow/unfollow them using a dedicated button. The same can be done from the `UserCard` context menu. In addition, the Profile component renders the User Feed, see ... for more details.

`UserCard`, `UserAvatar` and `UserIdentifier` components are used throughout the application where User presentation is needed. For example, a `UserCard` is rendered for each Author/User search result and UserIdentifier is shown in Comment/Post headers, so that the users are easier to identify.

User's Connections (Followed/Followers/Friend) are exposed via a `UserConnectionsProvider`. And a WS "events" handler is added so that the User or Connection-related updates trigger UI re-renders.

With the User-related functionality discussed, the following section discusses the actual code that controls how the User accounts are created and validated.

6.5 Account Creation and Authentication

This section presents the account creation flow and the following authentication mechanisms.

The platform can not be accessed by Anonymous Users, hence, every User has to have an account and a valid login to use the system.

Bibliography

[1] *Music Streaming Market Decade Insights*. URL: https://www.businessofapps.com/data/music-streaming-market/ (visited on 11/17/2025).

[2] Peter J. Rentfrow. *The Role of Music in Everyday Life: Current Directions in the Social Psychology of Music*. 2012. URL: https://compass.onlinelibrary.wiley.com/doi/abs/10.1111/j.1751-9004.2012.00434.x (visited on 03/15/2025).

[3] *Sharing Music: Social and Communal Aspects of Concert-Going*. URL: https://ojs.meccsa.org.uk/index.php/netknow/article/view/428/250 (visited on 11/17/2025).

[4] *Google Forms*. URL: https://docs.google.com/forms/u/0/ (visited on 04/05/2025).

[5] *Spotify Popularity*. URL: https://simplebeen.com/music-streaming-statistics/?utm_source=chatgpt.com (visited on 04/15/2025).

[6] *Audio Usage Statistics 2024*. URL: https://www.ifpi.org/wp-content/uploads/2023/12/IFPI-Engaging-With-Music-2023_full-report.pdf (visited on 03/15/2025).

[7] *Spotify Jam*. URL: https://support.spotify.com/cz/article/jam/ (visited on 03/16/2025).

[8] *Spotify Friend Activity*. URL: https://support.spotify.com/cz/article/friend-activity/ (visited on 03/16/2025).

[9] *Spotify Party Announcement*. URL: https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features/ (visited on 03/16/2025).

[10] *Spotify Party*. URL: https://support.spotify.com/us/article/listening-party/ (visited on 03/16/2025).

[11] *Spotify Chat*. URL: https://newsroom.spotify.com/2025-08-26/introducing-messages-a-new-way-to-share-what-you-love-on-spotify-with-friends-and-family/ (visited on 11/17/2025).

[12] *VK Music*. URL: https://music.vk.com/ (visited on 11/17/2025).

[13] *SoundCloud Comments*. URL: https://help.soundcloud.com/hc/en-us/articles/115003566008-Commenting-basics (visited on 03/16/2025).

[14] *SoundCloud Reactions*. URL: https://help.soundcloud.com/hc/en-us/articles/31039497560731-Reactions (visited on 03/16/2025).

[15] *Soundcloud Reposts*. URL: https://help.soundcloud.com/hc/en-us/articles/115003567488-Reposting-tracks-or-playlists (visited on 03/16/2025).

[16] *Rate Your Music*. URL: https://rateyourmusic.com/discover (visited on 03/25/2025).

[17] *Instagram Audio*. URL: https://help.instagram.com/664508917489819?helpref=faq_content (visited on 03/25/2025).

[18] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. URL: https://dl.acm.org/doi/10.5555/561543 (visited on 05/05/2025).

[19] *Multipurpose Internet Mail Extensions Part Two: Media Types. 'octet-stream'*. URL: https://www.rfc-editor.org/rfc/rfc2046#section-4.5.1 (visited on 11/30/2025).

[20] *Nginx pseudo-streaming implementation*. URL: https://nginx.org/en/docs/http/ngx_http_mp4_module.html (visited on 11/30/2025).

[21] *HTTP Range Requests*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Range_requests (visited on 12/07/2025).

[22] Issing Jochen Färber Nikolaus Döhla Stefan. *Adaptive progressive download based on the MPEG-4 file format*. URL: https://link.springer.com/article/10.1631/jzus.2006.AS0106 (visited on 11/30/2025).

[23] *The MPEG-DASH Standard for Multimedia Streaming Over the Internet*. URL: https://ieeexplore.ieee.org/abstract/document/6077864 (visited on 11/28/2025).

[24] *AWS CDN Caching*. URL: https://aws.amazon.com/caching/ (visited on 12/14/2025).

[25] Beng Chin Ooi Quang Hieu Vu Mihai Lupu. *Peer-to-Peer Computing: Principles and Applications*. URL: https://books.google.

tt/books?id=kd8_AAAAQBAJ&printsec=frontcover&source=gbs_vpt_read#v=onepage&q&f=false (visited on 01/21/2026).

[26] *Basic Authentication*. URL: https://datatracker.ietf.org/doc/html/rfc7617 (visited on 03/19/2025).

[27] *Mozilla Caching Behaviour Bug Tracker*. URL: https://bugzilla.mozilla.org/show_bug.cgi?id=300002 (visited on 11/01/2025).

[28] *Mozilla Caching Behaviour Documentation*. URL: https://support.mozilla.org/en-US/kb/delete-browsing-search-download-history-firefox (visited on 11/01/2025).

[29] *Overview of TLS termination and end to end TLS with Application Gateway*. URL: https://learn.microsoft.com/en-us/azure/application-gateway/ssl-overview (visited on 01/21/2026).

[30] *Session Authentication*. URL: https://docs.djangoproject.com/en/5.2/topics/http/sessions/ (visited on 03/19/2025).

[31] *OAuth2*. URL: https://datatracker.ietf.org/doc/html/rfc6749 (visited on 03/21/2025).

[32] *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*. URL: https://datatracker.ietf.org/doc/html/rfc6202 (visited on 11/30/2025).

[33] *Comparison of Server to Client Communication Methods*. URL: https://www.diva-portal.org/smash/get/diva2:1133465/FULLTEXT01.pdf?ref=hackernoon.com (visited on 03/29/2025).

[34] *Server Sent Events*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events (visited on 05/02/2025).

[35] *Django Channels*. URL: https://channels.readthedocs.io/en/latest/ (visited on 03/19/2025).

[36] *WebSockets*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API (visited on 05/02/2025).

[37] *Http Caching*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching (visited on 05/02/2025).

[38] *Push API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Push_API (visited on 05/02/2025).

[39] *Generic Event Delivery Using HTTP Push*. URL: https://datatracker.ietf.org/doc/html/rfc8030 (visited on 11/30/2025).

[40] *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (visited on 11/30/2025).
[41] *FastAPI*. URL: https://fastapi.tiangolo.com/ (visited on 03/18/2025).
[42] *Pydantic*. URL: https://docs.pydantic.dev/latest/ (visited on 05/01/2025).
[43] *FastAPI Features*. URL: https://fastapi.tiangolo.com/features (visited on 03/20/2025).
[44] *FastAPI Websockets*. URL: https://fastapi.tiangolo.com/advanced/websockets/ (visited on 12/01/2025).
[45] *StackOverflow 2024 Developers Survey*. URL: https://survey.stackoverflow.co/2024/technology (visited on 12/01/2025).
[46] *Django*. URL: https://www.djangoproject.com/ (visited on 03/18/2025).
[47] *DRF*. URL: https://www.django-rest-framework.org/ (visited on 03/19/2025).
[48] *Django Overview*. URL: https://www.djangoproject.com/start/overview/ (visited on 12/07/2025).
[49] *Django Async Support*. URL: https://docs.djangoproject.com/en/5.2/topics/async/ (visited on 01/30/2025).
[50] *Flask*. URL: https://flask.palletsprojects.com/en/stable/ (visited on 03/19/2025).
[51] *SocketIO*. URL: https://socket.io/ (visited on 12/01/2025).
[52] *Web Service Efficiency at Instagram with Python*. URL: https://instagram-engineering.com/web-service-efficiency-at-instagram-with-python-4976d078e366 (visited on 12/07/2025).
[53] *Oracle Price List*. URL: https://www.oracle.com/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf (visited on 11/30/2025).
[54] *PostgreSQL Full Text Search*. URL: https://www.postgresql.org/docs/current/textsearch.html (visited on 12/07/2025).
[55] *Elasticsearch*. URL: https://www.elastic.co/elasticsearch (visited on 01/21/2026).
[56] *Sphinx Search Engine*. URL: sphinxsearch.com (visited on 01/21/2026).

[57] *PostgreSQL*. URL: https://www.postgresql.org/about/ (visited on 03/26/2025).
[58] *Django PostgreSQL Support*. URL: https://docs.djangoproject.com/en/5.2/ref/contrib/postgres/ (visited on 03/26/2025).
[59] *Postgres UUID Type*. URL: https://www.postgresql.org/docs/current/datatype-uuid.html (visited on 12/01/2025).
[60] *UUIDv7*. URL: https://www.thenile.dev/blog/uuidv7 (visited on 01/21/2026).
[61] *Django Templates*. URL: https://docs.djangoproject.com/en/5.2/ref/templates/language/ (visited on 01/30/2025).
[62] *Single Page Application*. URL: https://developer.mozilla.org/en-US/docs/Glossary/SPA (visited on 12/07/2025).
[63] *Clay Library*. URL: https://github.com/nicbarker/clay (visited on 12/01/2025).
[64] *Yew Framework*. URL: https://github.com/yewstack/yew (visited on 12/01/2025).
[65] *Framework Usage*. URL: https://2024.stateofjs.com/en-US/libraries/ (visited on 05/01/2025).
[66] *React*. URL: https://react.dev/ (visited on 03/30/2025).
[67] *React Developer Tools*. URL: https://react.dev/learn/react-developer-tools (visited on 05/01/2025).
[68] *StackOverflow React*. URL: https://stackoverflow.com/questions/tagged/reactjs?tab=Newest (visited on 12/07/2025).
[69] *MVC*. URL: https://developer.mozilla.org/en-US/docs/Glossary/MVC (visited on 03/18/2025).
[70] *uv*. URL: https://docs.astral.sh/uv/ (visited on 01/24/2026).
[71] *Daphne GitHub Page*. URL: https://github.com/django/daphne (visited on 01/24/2026).
[72] *django-filter*. URL: https://django-filter.readthedocs.io/en/stable/ (visited on 01/24/2026).
[73] *Redis*. URL: https://redis.io/ (visited on 12/07/2025).
[74] *dj-rest-auth*. URL: https://pypi.org/project/dj-rest-auth/ (visited on 01/24/2026).
[75] *MPEG-DASH*. URL: https://www.mpeg.org/standards/MPEG-DASH/ (visited on 05/05/2025).
[76] *HLS*. URL: https://developer.apple.com/streaming/ (visited on 05/05/2025).

[77] *Packaging explanation*. URL: https://shaka-project.github.io/shaka-packager/html/documentation.html (visited on 12/13/2025).
[78] *FFMPEG*. URL: https://ffmpeg.org/ (visited on 05/05/2025).
[79] *Shaka Packager*. URL: https://shaka-project.github.io/shaka-packager/html/ (visited on 12/13/2025).
[80] *Shaka Player*. URL: https://github.com/shaka-project/shaka-player (visited on 05/08/2025).
[81] *HTTP Live Streaming (HLS) authoring specification for Apple devices. Bitrate recommendations, section 2.25.* URL: https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices#Bitrate-recommendations-for-Ambisonics-are-as-follows (visited on 12/13/2025).
[82] *EBU Evaluations of Mutlichannel Audio Codecs*. URL: https://tech.ebu.ch/docs/tech/tech3324.pdf (visited on 12/14/2025).
[83] *Perceived Audio Quality for Streaming Stereo Music*. URL: https://dl.acm.org/doi/pdf/10.1145/2647868.2655025 (visited on 12/13/2025).
[84] *BACHELOR THESIS Perceived Audio Quality in Ogg Vorbis and AAC in Compressed Popular Music in Bit Rates between 96 kbit/s, 128 kbit/s and 192 kbit/s*. URL: https://www.diva-portal.org/smash/get/diva2:1030609/FULLTEXT02.pdf (visited on 12/14/2025).
[85] *Spotify Audio Quality*. URL: https://support.spotify.com/cz/article/audio-quality/ (visited on 12/14/2025).
[86] *Web audio codec guide*. URL: hhttps://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Audio_codecs#aac_advanced_audio_coding (visited on 12/14/2025).
[87] *python-magic Package*. URL: https://pypi.org/project/python-magic/ (visited on 05/07/2025).